P1370R1: Generic numerical algorithm development with(out) `numeric_limits`

Table of Contents

1 Feedback

1.1 Kona 2019

1.1.1 SG6 (Numerics)

Revise with addtional `safe_min` value (name subject to future bikeshedding)?

SF	F	N	A	SF
5	0	2	0	0

Revise with explanation of distinction betweeen the various constants? (Authors' note: This refers specifically to two constants: the minimum positive normalized floating-point number (what Fortran calls `TINY`), and the minimum positive floating-point number s such that 1/s is finite (what LAPACK calls `SFMIN`).)

SF	F	N	A	SF
5	2	0	0	0

Merge into a revision of P0437r2? (Authors' note: Intent is that this paper should first be revised and resubmitted, and then should be rebased on top of the latest version of P0437.)

SF	F	N	A	SF
5	2	0	0	0

1.2 SG18 (LEWG Incubator)

The LEWG-I minutes were not posted, but the authors remember the following discussion points.

1. Suggested name for what Fortran calls `TINY`: `min_normal_v` (a term of art from IEEE 754).
2. Suggested name for what LAPACK calls `sfmin`: `reciprocal_overflow_threshold_v` (there was another, more concise name, but I don't recall it and don't have the minutes). The group strongly objected to names with the word "safe" in them.

In discussion, one LEWG-I participant expressed a preference that the new traits proposed by P0437 should come in `has_${PROPERTY}_v<T>`, `${PROPERTY}_v` pairs. They prefer the current `numeric_limits`-like behavior, in which `${PROPERTY}_v<T>` exists but has some arbitrary value if `has_${PROPERTY}_v<T>` is false, to the approach in which `${PROPERTY}_v<T>` does not exist if `T` does not have the property in question. SG18 did not vote on this topic, but this feedback could be helpful for the next revision of P0437. In particular, the last revision of P0437 before the 2019 Kona meeting proposes that use of `${PROPERTY}_v<T>` be ill-formed if if `T` does not have the property in question. Section 6 of P0437R1 suggests a general mechanism, `value_exists<Trait, T>`, that would replace `has_${PROPERTY}_v<T>`. We prefer P0437R1's approach, since it prevents bugs at compile time. However, our proposal should work with both approaches.

2 Proposal

1. In the next revision to P0437[1], whatever trait replaces `numeric_limits<T>::min` should always give the minimum finite value of `T`. For floating-point types, it should give the same value as `numeric_limits<T>::lowest()` does now.
2. The actual current value of `numeric_limits<T>::min()` for floating-point types `T` is the minimum positive normalized value of `T`. This is useful for writing generic numerical algorithms, but the name "min" is confusing. We propose calling this trait `min_normal_v<T>`, based on the IEEE 754[2] term of art.
3. Introduce a new trait, `reciprocal_overflow_threshold_v<T>`, for the smallest positive floating-point number such that one divided by that number does not overflow. This has differed from `min_normal_v<T>` for historical floating-point types, and has different uses than `min_normal_v<T>`.

3 Introduction

P0437R1[1] proposes to "[b]reak the monolithic `numeric_limits` class template apart into a lot of individual trait templates, so that new traits can be added easily." We see this as an opportunity both to revise the definitions of these traits, and to consider adding new traits.

3.1 Smallest positive normalized value

An example of existing traits needing revision is the current definition of `numeric_limits<T>::min`, which is inconsistent for integer versus floating-point types `T`. For integer types, this function behaves as expected; it returns the minimum finite value. For signed integer types, this is the *negative* value of largest magnitude, and for unsigned integer types, it is zero. However, for a floating-point type, it returns the type's smallest positive normalized value. This is confusing in two different ways:

1. It is positive, so it is not the least finite floating-point value, namely `-numeric_limits<T>::max()`.
2. It is normalized, so if the implementation of floating-point arithmetic has subnormal numbers, smaller positive floating-point numbers of type `T` could exist.[3]

This surprising behavior could lead to bugs when writing algorithms meant for either integers or floating-point values. Nevertheless, the actual value is useful in practice, as we will show below.

3.2 Reciprocal overflow threshold

An example of a new trait to consider is the smallest positive floating-point number such that one divided by that number does not overflow. We propose calling this trait `reciprocal_overflow_threshold_v<T>`. As we will show below, floating-point types exist for which this trait's value differs from that of `min_normal_v<T>`. This new trait is also useful in practice.

4 Example: the LAPACK linear algebra library

In this section, we will show that the two proposed traits are useful constants for writing generic numerical algorithms, and that they have distinct uses. The LAPACK[4] Fortran linear algebra library uses both values and distinguishes between them.

To clarify: *Numerical algorithms* use floating-point numbers as approximations to real numbers, to do the kinds of computations that scientists, engineers, and statisticians often find useful. *Generic numerical algorithms* are written to work for different kinds of floating-point number types.

4.1 LAPACK is a library of generic numerical algorithms

LAPACK is a Fortran library, but it takes a "generic" approach to algorithms for different data types. LAPACK implements algorithms for four different data types:

- Single-precision real (S)
- Double-precision real (D)
- Single-precision complex (C)
- Double-precision complex (Z)

LAPACK does not rely on Fortran generic procedures or parameterized derived types, the closest analogs in Fortran to C++ templates. However, most of LAPACK's routines are implemented in such a way that one could generate all four versions automatically from a single "template."[5] As a result, we find LAPACK a useful analog to a C++ library of generic numerical algorithms, written using templates and traits classes. Numerical algorithm developers who are not C++ programmers have plenty of experience writing generic numerical algorithms. See, for example, "Templates for the Solution of Linear Systems,"[6] where "templates" means "recipes," not C++ templates. Thus, it should not be surprising to find design patterns in common between generic numerical algorithms not specifically using C++, and generic C++ libraries. In fact, our motivating examples will come from LAPACK's "floating-point traits" routines.

LAPACK's "generic" approach means that algorithm developers need a way to access floating-point arithmetic properties as a function of data type, just as if a C++ developer were writing an algorithm templated on a floating-point type. Many linear algebra algorithms depend on those properties to avoid unwarranted overflow or underflow, and to get accurate results. As a result, LAPACK provides the `SLAMCH` and `DLAMCH` routines, that return machine parameters for the given real floating-point type (single-precision real resp. double-precision real). (One can derive from these the properties for corresponding complex numbers.)

LAPACK routines have a uniform naming convention, where the first letter indicates the data type. LAPACK developers refer to the "generic" algorithm by omitting the first letter. For example, `_GEQRF` represents the same QR factorization for all data types for which it is implemented, in this case, `SGEQRF`, `DGEQRF`, `CGEQRF`, and `ZGEQRF`. Hence, we refer to the "floating-point traits" routines `SLAMCH` and `DLAMCH` generically as `_LAMCH`.

`_LAMCH` was designed to work on computers that may have non-IEEE-754 floating-point arithmetic. Older versions of the routine would actually compute the machine parameters. This is what LAPACK 3.1.1 does.[7]

More recent versions of LAPACK, including the most recent version, 3.8.0, rely on Fortran intrinsics to get the values of most of the machine parameters.[8]

_LAMCH thus offers functionality analogous to numeric_traits, for different real floating-point types. LAPACK's authors chose this functionality specifically for the needs of linear algebra algorithm development. _LAMCH gives developers the following constants:

- eps: relative machine precision
- sfmin: safe minimum, such that 1/sfmin does not overflow
- base: base of the machine
- prec: eps*base
- t: number of (base) digits in the mantissa
- rnd: 1.0 when rounding occurs in addition, 0.0 otherwise[9]
- emin: minimum exponent before (gradual) underflow
- rmin: underflow threshold - base**(emin-1)
- emax: largest exponent before overflow
- rmax: overflow threshold - (base**emax)*(1-eps)

4.2 How LAPACK uses our two proposed traits

Our proposed min_normal_v<T> corresponds to the underflow threshold rmin, and our reciprocal_overflow_threshold_v<T> corresponds to the "safe minimum" sfmin. LAPACK uses the "safe minimum" more often than the underflow threshold, but it does use both values.

LAPACK uses the safe minimum in algorithms (e.g., equilibration and balancing) that scale the rows and/or columns of a matrix to improve accuracy of a subsequent factorization. In the process of improving accuracy, one would not want to divide by too small of a number, and thus cause underflow that is not warranted by the user's data. For example, see DRSCL, DLADIV, and ZDRSCL. (The documentation of LAPACK's _LABAD routine explains how LAPACK uses this routine to work around surprising behavior at the extreme exponent ranges of certain floating-point systems.)

The safe minimum also helps in LAPACK's LU factorization with complete pivoting (see e.g., ZGETC2). If LAPACK finds a pivot U(k, k) that is too small in magnitude, it replaces the pivot with a tiny real number. This helps the factorization finish, which is an important goal for LU with complete pivoting (unlike the usual LU factorization with partial pivoting _GETRF, which stops on encountering a zero pivot). LAPACK derives the value for this tiny real number from the safe minimum, since LU factorization must divide numbers in the matrix by the pivot.

LAPACK uses the underflow threshold quite a bit in its tests. In its actual code, it uses this value when computing the eigenvalues of a real symmetric tridiagonal matrix (the _LARRD auxiliary routine, called from _STEMR).

4.3 How LAPACK computes the two traits

In the most recent version of LAPACK, 3.8.0, LAPACK uses the Fortran intrinsic function TINY for the underflow threshold, and derives the "safe minimum" sfmin from that value and the overflow threshold (Fortran intrinsic HUGE) as follows:

```
         sfmin = tiny(zero)
         small = one / huge(zero)
         IF( small.GE.sfmin ) THEN
*
*           Use SMALL plus a bit, to avoid the possibility of rounding
*           causing overflow when computing  1/sfmin.
```

```
*
            sfmin = small*( one+eps )
         END IF
         rmach = sfmin
```

Here is the C++ equivalent:

```
template<class T>
T safe_minimum (const T& [redacted]) {
  constexpr T one (1.0);
  constexpr T eps = std::numeric_limits<T>::epsilon();
  constexpr T tiny = std::numeric_limits<T>::min();
  constexpr T huge = std::numeric_limits<T>::max();
  constexpr T small = one / huge;
  T sfmin = tiny;
  if (small >= tiny) {
    sfmin = small * (one + eps);
  }
  return sfmin;
}
```

4.4 The two traits' values can differ

For IEEE 754 `float` and `double`, the `IF` branch never gets taken. (LAPACK was originally written to work on computers that did not implement IEEE 754 arithmetic, so the extra branches may have made sense for earlier computer architectures. They are also useful as a conservative check of floating-point properties.) Thus, for `T=float` and `T=double`, `sfmin` always equals `numeric_limits<T>::min()`. However, several historical floating-point formats have the property that one divided by the underflow threshold would overflow. Table 1 in Prof. William Kahan's essay[10] gives examples. In general, this is possible whenever the absolute value of the underflow threshold exponent is greater than the overflow threshold exponent. Whenever this property holds, LAPACK's `sfmin` is larger than the minimum normalized floating-point value.

WG21 has seen proposals both to standardize new number formats, and to standardize ways for users to add support for their own number formats. Furthermore, WG21 has seen proposals for generic linear algebra (e.g., P1385) and other algorithms useful for machine learning. This makes it critical for C++ developers, including Standard Library developers, to have Standard Library support for writing type-independent floating-point algorithms. Ignoring the two proposed traits puts these developers at risk, especially for algorithms that rescale data to improve accuracy (a common feature when solving linear systems, for example). Such algorithms could cause infinities and/or Not-a-Numbers that are not warranted by the data, unless they use the two proposed traits.

5 Conclusion

We (the authors) have experience as developers and users of a C++ library of generic numerical algorithms, namely Trilinos.[11] Many other such libraries exist, including Eigen[12]. We also use the LAPACK library extensively, and have some experience modifying LAPACK algorithms.[13] We use and write traits classes that sometimes make use of `numeric_limits`. While we have found `numeric_limits` useful, we think it could benefit from the following changes:

1. Split out different traits into separate traits classes (the task of P0437).
2. Replace `numeric_limits<T>::min` for floating-point types `T` with the new trait `min_normal_v<T>`.
3. Add a new trait `reciprocal_overflow_threshold_v<T>` for floating-point types `T`, whose value is the smallest positive `T` such that one divided by the value does not overflow.

Our thanks to Walter Brown for P0437, and for helpful discussion and advice.

6 Funding and disclaimer

Sandia National Laboratories is a multi-mission laboratory managed and operated by National Technology and Engineering Solutions of Sandia, LLC., a wholly owned subsidiary of Honeywell International, Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525. This paper describes objective technical results and analysis. Any subjective views or opinions that might be expressed in the paper do not necessarily represent the views of the U.S. Department of Energy or the United States Government.

Footnotes:

[1] Walter E. Brown, "Numeric Traits for the Standard Library," P0437R1, Nov. 2018. Available online: wg21.link/p0437r1.

[2] *IEEE 754* is the Institute of Electrical and Electronics Engineers' standard for binary floating-point computation. The standard first came out in 1985, and the latest revision was released in 2008. (The IEEE 754 Floating Point Standards Committee approved a new draft on 20 July 2018, as reported by Committee member James Demmel over e-mail.)

[3] Some systems have settings that change the behavior of floating-point arithmetic, in order to avoid subnormal numbers. These options include "flush to zero," where arithmetic results that should produce a subnormal instead result in zero, and "denormals are zero," where a subnormal input to arithmetic operations is assumed to be zero. These options exist in part because some hardware implementations of floating-point arithmetic handle subnormal numbers very slowly.

[4] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen, "LAPACK Users' Guide," 3rd ed., Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1999.

[5] J.J. Dongarra, oral history interview by T. Haigh, 26 Apr. 2004, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA; available from http://history.siam.org/oralhistories/dongarra.htm.

[6] See, for example, R. Barrett, M. Berry, T. F. Chan, J. Demmel, J. Donato, J. Dongarra, V. Eijkhout, R. Pozo, C. Romine, and H. Van der Vorst, "Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods," 2nd Edition, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1994. "Templates" in the title does not mean C++ templates; it means something more like "design patterns."

[7] For example, here is the implementation of `DLAMCH` in LAPACK 3.1.1: http://www.netlib.org/lapack/explore-3.1.1-html/dlamch.f.html

[8] See, for example, the much shorter implementation of `DLAMCH` in LAPACK 3.8.0: http://www.netlib.org/lapack/explore-html/d5/dd4/dlamch_8f_a06d6aa332f6f66e062e9b96a41f40800.html#a06d6aa332f6f66e062e9b96a41f40800

[9] "Otherwise" here means that addition chops instead of rounds.

[10] William Kahan, "Why do we need a floating-point standard?", Feb. 12, 1981. Available online: https://people.eecs.berkeley.edu/~wkahan/ieee754status/why-ieee.pdf (last accessed Mar. 5, 2019).

[11] M. A. Heroux et al., "An overview of the Trilinos project," ACM Transactions on Mathematical Software, Vol. 31, No. 3, Sep. 2005, pp. 397-423; M. A. Heroux and J. M. Willenbring, "A New Overview of The Trilinos Project," Scientific Programming, Vol 20, No. 2, 2012, pp. 83-88. See also: Trilinos' GitHub site, and E. Bavier, M. Hoemmen, S. Rajamanickam, and Heidi Thornquist, "Amesos2 and Belos: Direct and Iterative Solvers for Large Sparse Linear Systems," Scientific Programming, Vol. 20, No. 3, 2012, pp. 241-255.

[12] Gaël Guennebaud, Benoît Jacob, et al., "Eigen v3," http://eigen.tuxfamily.org, 2010 [last accessed Nov. 2018]. See also Eigen's documentation for "Using custom scalar types": http://eigen.tuxfamily.org/dox/TopicCustomizing_CustomScalar.html.

[13] See e.g., J. W. Demmel, M. Hoemmen, Y. Hida, and E. J. Riedy, "Nonnegative Diagonals and High Performance on Low-Profile Matrices from Householder QR," SIAM J. Sci. Comput., Vol. 31, No. 4, 2009, pp. 2832-2841. The authors later found out via a Matlab bug report that these changes to LAPACK's Householder reflector computation had subtle rounding error issues that broke one of LAPACK's dense eigensolver routines, so we ended up backing them out.

Date: 10 Mar 2019

Author: Mark Hoemmen (mhoemme@sandia.gov) and Damien Lebrun-Grandie (qdi@ornl.gov)

Created: 2019-03-10 Sun 19:35

Emacs 25.1.1 (Org mode 8.2.10)

Validate